

1. Introduction

Plant diseases significantly impact agricultural productivity and food security. Early and accurate detection of plant diseases using computer vision can assist farmers in taking timely corrective actions.

This project presents an **end-to-end machine learning system** for plant disease classification using the **PlantVillage dataset**, progressing from exploratory data analysis to centralized learning, federated learning, and finally model monitoring and deployment.

The focus is not only on accuracy, but also on **system design, scalability, privacy, and reproducibility**, aligning with real-world ML engineering practices.

2. Dataset Description

The **PlantVillage dataset** consists of labeled leaf images covering **38 plant disease classes**, including healthy and diseased samples across multiple crop species.

Key characteristics:

- RGB images
- High intra-class similarity
- Class imbalance in some disease categories
- Standard benchmark dataset for plant disease detection

The dataset was used consistently across all stages to ensure fair comparison.

3. Week 1 – Exploratory Data Analysis (EDA)

In Week 1, exploratory analysis was performed to understand dataset structure and challenges.

Key observations

- Large number of classes (38) increases classification complexity
- Visual similarity between certain diseases
- Class imbalance can bias model learning
- Images are clean but vary in texture and color patterns

EDA helped guide later architectural and evaluation decisions.

4. Week 2 – Classical Machine Learning (SVM Baseline)

A **Support Vector Machine (SVM)** model was implemented as a classical ML baseline.

Approach

- Images were preprocessed and flattened
- Dimensionality reduction was applied
- SVM trained on extracted features

Outcome

- Provided a meaningful baseline
- Performance limited due to:
 - High dimensional image space
 - Loss of spatial information
- Motivated the need for deep learning approaches

5. Week 3 – Centralized Deep Learning (CNN)

A **Convolutional Neural Network (CNN)** was trained in a centralized manner.

Model Architecture

- Convolution → Pooling blocks
- Fully connected layers
- Softmax output (38 classes)

Training Setup

- Centralized dataset
- Standard train/validation split
- Cross-entropy loss
- Adam optimizer

Results

- Strong improvement over SVM

- Validation accuracy stabilized with 90%+
- Signs of overfitting in later epochs

This stage established a strong supervised learning baseline.

6. Week 4 – Federated Learning with FedAvg

To simulate **privacy-preserving distributed learning**, the centralized pipeline was converted into a **Federated Learning (FL)** system.

Federated Setup

- Dataset split into **3 disjoint client shards**
- Each client trained locally for a small number of epochs
- No raw data sharing between clients
- Model updates aggregated using **Federated Averaging (FedAvg)**

Federated Training Process

1. Server initializes global model
2. Global weights sent to clients
3. Clients train locally
4. Clients return updated weights
5. Server aggregates weights using weighted average
6. Global model evaluated on held-out test set

Results

Federated Round	Global Test Accuracy
Round 1	~63%
Round 2	~91%
Round 5	~93%
Round 10	~ 94.3%

Key Insight

Federated learning achieved **comparable or better generalization** than centralized training due to:

- Regularization effect of weight averaging

- Reduced client-specific overfitting
- Smoother convergence behavior

7. Week 5 – Model Persistence, Monitoring, and Deployment

Week 5 focused on **ML system usability rather than training**.

7.1 Model & Metric Persistence

- Global federated metrics saved to JSON
- One entry per federated round
- Enables offline analysis and reproducibility

7.2 Streamlit Monitoring Dashboard

A **Streamlit web application** was built to visualize federated training behavior.

Features

- Loads saved federated metrics
- Plots global accuracy vs federated rounds
- Displays final model performance
- No retraining required

Purpose

- Separates training from analysis
- Mimics real ML monitoring workflows
- Improves observability of distributed training

Deployment

- Deployed via **Streamlit Cloud**
- Connected directly to GitHub repository
- Fully reproducible and public

8. System Design Perspective

This project transitions from **experiments** → **system**:

Stage	Focus
EDA	Understanding data
SVM	Baseline modeling
CNN	Centralized learning
Federated Learning	Privacy & scalability
Streamlit App	Monitoring & usability

This mirrors real-world ML pipelines where:

- Training is decoupled from deployment
- Metrics are logged, not printed
- Models are inspected post-training

9. Conclusion

This project demonstrates a **complete end-to-end ML system** for plant disease classification, progressing from classical ML to deep learning, federated learning, and deployment.

Beyond accuracy, the system emphasizes:

- Privacy awareness
- Distributed training
- Observability
- Reproducibility

The final outcome aligns closely with how **real ML systems are built, monitored, and deployed in production**.